

Dahlia Type System

Charlotte W.

July 19, 2026

1 Preamble

This document defines the formal type system for the Dahlia language. Dahlia is a tiny low-level, memory-safe programming language utilizing a static type system with type inference based on Hindley-Milner and Algorithm W.

2 Data Types

- **i8-i64**: A sized integer type (`i8-i64`) with specified bit width. Unsigned integers are denoted with a `u` rather than an `i` prefix.
- **bool**: A boolean type (`bool`) representing true or false values.
- **char**: A single character type (`char`) (8 bits wide).
- **str**: A string type (`str`) representing a sequence of characters.
- **Str**: A boxed string type (`Str`) representing a heap-allocated string.
- **Array**: A fixed-size array type (`[T; N]`) representing a collection of elements of type `T` with size `N`.

3 Constants

Constants are used to define values that cannot be changed during program execution. They are defined using: `const <name>: <type> = <value>`

4 How to Read the Rules

Each rule has one or more premises above the line and a conclusion below it. If every premise holds, then the conclusion is valid.

- Γ is the typing context, mapping names to types.
- τ and τ_r range over types.
- e ranges over expressions, and b ranges over blocks of statements.
- $\Rightarrow$ means a form synthesizes its type from the term itself.
- $\Leftarrow$ means a form is checked against an expected type.
- $\Rightarrow$ is used for statements and declarations that update or preserve the context.
- Items listed as "for each" or written with subscripts such as e_i must hold for every matching element.

5 Generic Type Syntax

Polymorphic types are written with square-bracket type parameters, for example `List[A]` or `Result[A, E]`. Type constructors are written in the constructor position, followed by their type arguments in square brackets. A type scheme is written as $\forall \alpha_1, \dots, \alpha_n. \tau$, where the quantified variables may be instantiated with fresh types during lookup.

$$\text{TYAPP} \quad \frac{\vdash \tau_1 \text{ type} \quad \dots \quad \vdash \tau_n \text{ type} \quad C \text{ is a type constructor of arity } n}{\vdash C[\tau_1, \dots, \tau_n] \text{ type}}$$

$$\text{TYScheme} \quad \frac{\vdash \tau \text{ type} \quad \alpha_1, \dots, \alpha_n \notin \text{ftv}(\Gamma)}{\Gamma \vdash \forall \alpha_1, \dots, \alpha_n. \tau \text{ scheme}}$$

This means that `List[A]` is well formed when `List` expects one type argument and `A` is itself a valid type, and similarly for constructors with more parameters such as `Result[A, E]`. A type scheme such as $\forall A. \text{List}[A] \rightarrow A$ is not a distinct runtime type; it is a polymorphic binding that may later be instantiated with fresh type variables.

6 Type Checking

The type system employs bidirectional type checking with synthesis and checking. The type system itself is based on Hindley-Milner, allowing for polymorphism and type inference.

Blocks are typed by the value of their terminal expression or explicit return; an empty block has type `void`.

- **Synthesis:** $\Gamma \vdash e \Rightarrow \tau$ means an expression synthesizes its type from the term in Γ .
- **Checking:** $\Gamma \vdash e \Leftarrow \tau$ means an expression is checked against an expected type τ in Γ .

The rules below are aligned with `GRAMMAR.md` and the current AST. Dahlia does not currently have lambda syntax or explicit type annotations on expressions, so the previous ANN and ABS rules do not apply.

Literal and variable rules

LIT $\frac{}{\Gamma \vdash \ell \Rightarrow \tau_\ell}$	VAR $\frac{x : \tau \in \Gamma}{\Gamma \vdash x \Rightarrow \tau}$	ARRAYLIT $\frac{\Gamma \vdash e_i \Leftarrow \tau \text{ for each } i \in 1..n}{\Gamma \vdash [e_1, \dots, e_n] \Rightarrow [\tau; n]}$
--	---	--

Expression rules

$\frac{\text{CALL} \quad \Gamma \vdash f \Rightarrow (\tau_1, \dots, \tau_n) \rightarrow \tau_r \quad \Gamma \vdash e_i \Leftarrow \tau_i \text{ for each } i \in 1..n}{\Gamma \vdash f(e_1, \dots, e_n) \Rightarrow \tau_r}$		
$\frac{\text{IF} \quad \Gamma \vdash c \Leftarrow \text{bool} \quad \Gamma \vdash b_1 \Rightarrow \tau \quad \Gamma \vdash b_2 \Rightarrow \tau}{\Gamma \vdash \text{if } c\{b_1\} \text{ else } \{b_2\} \Rightarrow \tau}$	$\frac{\text{ADDR} \quad \Gamma \vdash e \Leftarrow \tau}{\Gamma \vdash \&e \Rightarrow *\tau}$	
$\frac{\text{DEREF} \quad \Gamma \vdash e \Rightarrow *\tau}{\Gamma \vdash *e \Rightarrow \tau}$		

Operator rules

ARITH

$$\frac{\Gamma \vdash e_1 \Leftarrow \tau \quad \tau \in \{\text{i8}, \text{i16}, \text{i32}, \text{i64}, \text{u8}, \text{u16}, \text{u32}, \text{u64}, \text{f32}, \text{f64}\}}{\Gamma \vdash e_1 \oplus e_2 \Rightarrow \tau}$$

CMP

$$\frac{\Gamma \vdash e_1 \Leftarrow \tau \quad \tau \in \{\text{i8}, \text{i16}, \text{i32}, \text{i64}, \text{u8}, \text{u16}, \text{u32}, \text{u64}, \text{f32}, \text{f64}\}}{\Gamma \vdash e_1 \diamond e_2 \Rightarrow \text{bool}}$$

EQ

$$\frac{\Gamma \vdash e_1 \Leftarrow \tau \quad \Gamma \vdash e_2 \Leftarrow \tau}{\Gamma \vdash e_1 == e_2 \Rightarrow \text{bool}}$$

UNARY

$$\frac{\Gamma \vdash e \Leftarrow \tau}{\Gamma \vdash -e \Rightarrow \tau}$$

NOT

$$\frac{\Gamma \vdash e \Leftarrow \text{bool}}{\Gamma \vdash !e \Rightarrow \text{bool}}$$

Statement rules

VARDECL

$$\frac{\Gamma \vdash e \Leftarrow \tau}{\Gamma \vdash \text{var } x : \tau = e \Rightarrow \Gamma, x : \tau}$$

CONSTDECL

$$\frac{\Gamma \vdash e \Leftarrow \tau}{\Gamma \vdash \text{const } x : \tau = e \Rightarrow \Gamma, x : \tau}$$

ASSIGN

$$\frac{x : \tau \in \Gamma \quad \Gamma \vdash e \Leftarrow \tau}{\Gamma \vdash x = e \Rightarrow \Gamma}$$

WHILE

$$\frac{\Gamma \vdash c \Leftarrow \text{bool} \quad \Gamma \vdash b \Rightarrow \Gamma}{\Gamma \vdash \text{while } c \{b\} \Rightarrow \Gamma}$$

FOR

$$\frac{\Gamma \vdash it \Rightarrow [\tau; n] \quad \Gamma, x : \tau \vdash b \Rightarrow \Gamma}{\Gamma \vdash \text{for } x : it \{b\} \Rightarrow \Gamma}$$

RETURN

$$\frac{\Gamma \vdash e \Leftarrow \tau_r}{\Gamma \vdash \text{return } e \Rightarrow \Gamma}$$

BREAK

$$\frac{}{\Gamma \vdash \text{break} \Rightarrow \Gamma}$$

Declarations and polymorphism

$$\begin{array}{c}
\text{INST} \\
\frac{x : \forall \alpha_1, \dots, \alpha_n. \tau \in \Gamma}{\Gamma \vdash x \Rightarrow \tau[\alpha_1 \mapsto \beta_1, \dots, \alpha_n \mapsto \beta_n]} \\
\\
\text{GEN} \\
\frac{\Gamma \vdash e \Rightarrow \tau \quad \vec{\alpha} = \text{ftv}(\tau) \setminus \text{ftv}(\Gamma)}{\Gamma \vdash e \rightsquigarrow \forall \vec{\alpha}. \tau} \\
\\
\text{VARDECL} \qquad \qquad \qquad \text{CONSTDECL} \\
\frac{\Gamma \vdash e \Leftarrow \tau}{\Gamma \vdash \text{var } x : \tau = e \Rightarrow \Gamma, x : \tau} \qquad \frac{\Gamma \vdash e \Rightarrow \tau \quad \sigma = \text{Gen}(\Gamma, \tau)}{\Gamma \vdash \text{const } x : \tau = e \Rightarrow \Gamma, x : \sigma} \\
\\
\text{FNDECL} \\
\frac{\Gamma, x_1 : \tau_1, \dots, x_n : \tau_n \vdash b \Rightarrow \tau_r \quad \sigma = \text{Gen}(\Gamma, \tau_1 \rightarrow \dots \rightarrow \tau_n \rightarrow \tau_r)}{\Gamma \vdash \text{fn } f(x_1 : \tau_1, \dots, x_n : \tau_n)\{b\} \Rightarrow \Gamma, f : \sigma}
\end{array}$$

- **FnDecl**: function parameters are monomorphic within the body; the function name may be generalized after the body type is known.
- **VarDecl**: mutable bindings remain monomorphic so later assignments preserve a single type.
- **ConstDecl**: immutable bindings may be generalized because their type does not change after binding.
- **StructDecl**: struct fields must have unique names and well-formed types.
- **EnumDecl**: enum variants must have unique names.
- **Generic types**: type constructors use square-bracket syntax such as `List[A]`; the constructor arity must match the number of supplied arguments.
- **Instantiate**: each lookup of a polymorphic binding replaces quantified variables with fresh type variables.
- **Generalize**: when binding a value, quantify only type variables that are free in the inferred type but not free in the environment.
- **Unify**: synthesis and checking meet through unification with occurs-check.